PROPOSTA DE AUTOMAÇÃO DE TESTES DE API

Pytest + Requests

Uma base reproduzível para verificar contratos de API, acelerar o retorno sobre mudanças e apoiar entregas com mais previsibilidade.

Problema da empresa Verificar continuamente se APIs respondem conforme o esperado, reduzindo a dependência de conferências manuais repetidas.
Solução demonstrada Testes automatizados em Python usando Pytest para organização e Requests para comunicação HTTP.
Escopo atual 7 cenários: autenticação, leitura, pesquisa, paginação e criação simulada de produtos na DummyJSON.

Documento de apoio ao projeto avaliativo de Automação de Testes de API

Base técnica: enunciado e material introdutório da disciplina; implementação presente no repositório do projeto.

1. Resumo executivo

A proposta aplica automação de testes no nível da API para verificar, por código, aspectos que sustentam o comportamento do sistema: status HTTP, dados essenciais da resposta, autenticação e parâmetros de consulta. O objetivo empresarial é ter um retorno repetível sobre alterações no back-end antes de depender de uma validação completa pela interface.

A ferramenta escolhida combina duas bibliotecas de Python. **Requests** envia as requisições HTTP. **Pytest** descobre, prepara e executa os cenários, organiza fixtures e mostra quais verificações passaram ou falharam. Essa separação permite reaproveitar a configuração HTTP e manter cada cenário legível. [1][2]

> **Resultado proposto: incorporar testes de API de execução rápida ao ciclo de desenvolvimento e conectá-los ao CI. Os testes atuais são uma demonstração de abordagem, não evidência de melhoria medida em produção.**

O problema descrito no trabalho

A empresa fictícia quer acelerar entregas e elevar a qualidade do software por meio de automação de API. O desafio não é apenas fazer uma chamada funcionar: a equipe precisa demonstrar critérios claros de validação, reutilização, manutenção, execução automatizada e capacidade de explicar as decisões técnicas.

Recomendação

Tratar este repositório como piloto. Primeiro executar e estabilizar os sete cenários contra a API de demonstração; depois apontá-los para um ambiente de teste controlado da empresa e ampliar os contratos cobertos. Só então considerar o resultado como uma etapa obrigatória do pipeline de entrega.

O que o protótipo cobre hoje

- Login e consulta do usuário autenticado por token Bearer.
- Listagem, consulta por identificador, busca e paginação de produtos.
- Envio de POST para demonstrar o formato de resposta de criação.
- Execução local pelo Pytest e fluxo de CI configurado no GitHub Actions.

2. Problema e proposta técnica

Testes feitos apenas pela interface dependem da tela, dos dados e de vários componentes ao mesmo tempo. Os testes de API verificam diretamente a conversa HTTP entre cliente e serviço. O material da disciplina destaca que eles permitem cobrir regras do back-end sem precisar repetir cada verificação em uma jornada visual completa.

Como o fluxo funciona

1. Preparar URL-base, credenciais de demonstração e timeout vêm da configuração.	**2. Requisitar** Requests envia método, rota, cabeçalhos, parâmetros e JSON.	**3. Validar** Pytest compara status HTTP e campos importantes do corpo.

No cenário de autenticação, o teste envia `POST /auth/login`, lê `accessToken` e faz `GET /auth/me` com `Authorization: Bearer `. A documentação da DummyJSON descreve esse fluxo e os campos de token. [3]

Por que Pytest + Requests

Parte	Papel	Benefício para o trabalho
Requests	Cliente HTTP para GET e POST; suporta sessão, cabeçalhos, parâmetros, JSON e timeout.	Centraliza comunicação HTTP em um cliente que os cenários reutilizam. [4]
Pytest	Runner que descobre e executa funções de teste e fornece fixtures.	Mantém preparação e limpeza separadas e facilita acrescentar cenários. [1]
Python	Linguagem do projeto e das verificações.	Permite manter os testes junto ao ecossistema de desenvolvimento Python.

A combinação é adequada quando a equipe trabalha com Python e aceita manter os testes em código. Ela não substitui testes de interface ponta a ponta, revisão de contrato nem validação de regras no domínio.

3. Arquitetura e organização do repositório

O desenho mantém a configuração do transporte separada das asserções. A fixture cria um cliente por cenário, fornece valores de ambiente e fecha a sessão ao final. O Pytest documenta fixtures como recursos explícitos e reutilizáveis para preparar contexto e limpeza. [1]

Arquivo / pasta	Responsabilidade	Ponto de manutenção
api/client.py	ApiClient cria uma requests.Session, define Accept e timeout, e oferece GET e POST. GET aceita token Bearer e query params.	Adicionar métodos ou cabeçalhos comuns uma vez, sem duplicar a montagem da URL.
tests/conftest.py	Fixtures api_client e demo_credentials usam variáveis de ambiente com valores de demonstração padrão.	Trocar URL ou credenciais sem editar os cenários. Pytest encontra fixtures comuns em conftest.py.
tests/test_auth.py	Dois casos positivos: login e consulta do usuário autenticado.	Adicionar casos de token inválido, expiração ou erro assim que houver contrato estável.
tests/test_products.py	Cinco casos: listagem, detalhe, paginação, busca e POST simulado.	Organizar novos endpoints por recurso e reutilizar cliente/fixtures.
pytest.ini / requirements.txt	Configuração de descoberta e dependências.	Padronizar instalação e execução para todas as pessoas do grupo.
.github/workflows/tests.yml	Workflow inicia em push, pull request ou execução manual; instala Python/dependências e roda Pytest.	Automatizar a checagem na integração contínua. A documentação do GitHub cobre esse fluxo para Python. [5]

Configuração considerada

```
DUMMYJSON_BASE_URL   -> https://dummyjson.com (padrão)
DUMMYJSON_USERNAME   -> usuário de demonstração
DUMMYJSON_PASSWORD   -> senha de demonstração
REQUEST_TIMEOUT      -> 10 segundos (padrão)
```

Requests não define timeout por padrão; o cliente do projeto usa um limite configurável para evitar aguardar indefinidamente uma resposta. [4]

4. Cenários implementados

A suíte contém sete cenários de integração. Cada um envia chamadas reais à API DummyJSON e valida a resposta. A contagem descreve cobertura do protótipo, não cobertura percentual do sistema da empresa.

ID	Cenário	Chamadas	Validações centrais
A1	Login bem-sucedido	POST /auth/login	HTTP 200; username esperado; id numérico; accessToken e refreshToken presentes.
A2	Usuário autenticado	POST /auth/login GET /auth/me	HTTP 200; id, username e email presentes/esperados no perfil.
P1	Listar produtos	GET /products	HTTP 200; lista não vazia; total, limit e skip coerentes.
P2	Consultar produto	GET /products/1	HTTP 200; id 1; título textual; preço positivo; categoria presente.
P3	Paginar produtos	GET /products?limit=5&skip;=10	HTTP 200; cinco itens; limit e skip iguais aos enviados.
P4	Buscar produtos	GET /products/search?q=phone	HTTP 200; coleção não vazia; total coerente; títulos textuais.
P5	Simular inclusão	POST /products/add	HTTP 201; id numérico; título e preço retornam como enviados.

Boas decisões de asserção

- Verificar o código HTTP e campos estáveis que importam para o cenário.
- Não comparar a resposta inteira quando pode conter dados variáveis ou não relevantes.
- Isolar o login dentro do próprio cenário de usuário autenticado para evitar ordem entre testes.
- Deixar explícito que o POST de produtos da DummyJSON é simulado e não persiste o registro. [6]

5. Como executar e apresentar a demonstração

Os testes consultam uma API externa, portanto precisam de acesso à internet. A documentação do repositório inclui instalação e comandos para Windows, macOS e Linux. O comando direto no PowerShell para executar toda a suíte é:

```
.\.venv\Scripts\python.exe -m pytest -v
```

Para focar apenas no primeiro cenário de login:

```
.\.venv\Scripts\python.exe -m pytest -v tests/test_auth.py::test_login_returns_access_token
```

Roteiro curto para a demonstração

- Apresentar o ApiClient e explicar como ele centraliza base URL, sessão e timeout.
- Mostrar `test_login_returns_access_token`: o corpo de credenciais, o status e os campos de token verificados.
- Mostrar `test_authenticated_user_endpoint_returns_logged_in_user`: token recebido, cabeçalho Bearer e asserções do perfil.
- Rodar a suíte, apontar o resumo do Pytest e explicar um resultado de sucesso e uma falha de asserção.
- Abrir `.github/workflows/tests.yml` e explicar quando a execução automática acontece.

Estado relatado: o primeiro teste de login passou quando executado anteriormente. Os cinco novos cenários de produtos e o segundo cenário de autenticação foram adicionados depois; a suíte ampliada ainda precisa ser executada para confirmar seus resultados.

Variáveis e dados

As credenciais de demonstração são configuráveis por ambiente. Não grave credenciais pessoais, tokens ou dados reais no repositório. Em CI, prefira secrets para credenciais de ambientes privados. Para o endpoint público e didático, os dados são fictícios e fornecidos pela própria DummyJSON. [3]

6. Benefícios esperados e evolução empresarial

A automação pode contribuir para qualidade e velocidade quando produz retorno rápido, repetível e acionável. O protótipo mostra a estrutura técnica; benefícios empresariais devem ser medidos durante um piloto, não presumidos a partir da demonstração.

Benefícios que a abordagem permite buscar

- **Feedback mais cedo:** detectar incompatibilidades de status ou campos enquanto o serviço é alterado.
- **Repetibilidade:** executar o mesmo conjunto localmente e no CI, sem remontar requisições manualmente.
- **Reutilização:** centralizar URL, sessão, timeout e autenticação em componentes comuns.
- **Histórico por mudança:** associar aprovação ou falha do workflow ao commit ou pull request.

Caminho de adoção

Etapa	Ação	Condição para avançar
1. Estabilizar o piloto	Executar os sete cenários e corrigir inconsistências. Registrar resultados e duração de cada execução.	Resultados repetíveis em várias execuções e entendimento do time sobre cada asserção.
2. Migrar para ambiente controlado	Apontar `DUMMYJSON_BASE_URL` para staging; usar contas e massa de dados de teste isoladas.	Ambiente e dados com responsáveis, reset/recriação documentados e credenciais protegidas.
3. Ampliar cobertura	Adicionar cenários negativos, validações de schema, autorização, erros de negócio e recursos prioritários.	Cada cenário associado a risco, requisito ou contrato de API.
4. Integrar com entrega	Usar workflow em pull request; publicar relatório JUnit e definir política de bloqueio após estabilizar.	Falhas atribuíveis e ação clara; não bloquear a entrega por indisponibilidade externa não controlada.

Métricas sugeridas para avaliar o piloto

Acompanhar duração da suíte; proporção de execuções interrompidas por instabilidade externa; falhas confirmadas por regressão; tempo até identificar e resolver a falha; endpoints e requisitos prioritários cobertos. Definir a linha de base antes do piloto e comparar resultados depois, sem inventar metas sem dados.

7. Limitações, riscos e recomendação

A DummyJSON é apropriada para aprendizado e prototipação, mas não representa o ambiente produtivo da empresa. Os seus dados e disponibilidade pertencem a um serviço público externo; alterações ou interrupções podem afetar a execução sem que o código do produto tenha mudado.

Limitações do protótipo

- Não mede impacto real em tempo de entrega, defeitos ou custos.
- Não valida uma API pertencente à empresa nem regras de negócio específicas dela.
- O endpoint `POST /products/add` simula inclusão e não persiste dados. [6]
- A suíte atual prioriza cenários de sucesso; precisa de respostas de erro estáveis para casos negativos robustos.
- O workflow foi configurado, mas ainda depende de validação no repositório GitHub e de acesso à API pública.

Riscos e respostas

Risco	Resposta recomendada
API pública fora do ar ou alterada	Usar DummyJSON para apresentação; nos gates de entrega, apontar para staging controlado ou separar testes de contrato externos.
Credenciais expostas	Usar usuários sintéticos; armazenar segredos da CI no recurso protegido do provedor.
Teste frágil por valor volátil	Validar tipos, presença e invariantes relevantes; não fixar timestamps, preços mutáveis ou ordem arbitrária.
Falso bloqueio de release	Estabilizar e categorizar falhas antes de tornar o job obrigatório; preservar logs e relatório de execução.

Recomendação à empresa: aprovar um piloto pequeno de testes de contrato de API em Pytest + Requests, conectado primeiro a staging e acompanhado por indicadores. Tornar a suíte um gate de release somente depois de demonstrar estabilidade, relevância e manutenção viável.

Referências

Materiais da disciplina

- Introdução à Automação de Testes de API.pdf (material de aula fornecido pelo grupo).
- Trabalho Avaliativo Automação de Testes de API.pdf (enunciado e critérios da atividade).

Documentação técnica consultada

[3] DummyJSON - Auth · [6] DummyJSON - Products
[1] Pytest - Fixtures · [4] Requests - Sessions e timeouts
[5] GitHub Actions - Python

8. Síntese para a decisão

O projeto demonstra a forma de transformar chamadas de API em verificações que podem ser repetidas e executadas em integração contínua. Pytest organiza os cenários e fixtures; Requests monta as chamadas HTTP e aplica timeout; o cliente do projeto concentra a configuração compartilhada.

Decisão sugerida: conduzir o piloto em staging, validar a suíte completa e acompanhar estabilidade e utilidade antes de torná-la uma barreira de publicação.

Próximas ações

- Executar os sete cenários e guardar o resultado inicial como linha de base.
- Confirmar o workflow no GitHub com secrets apenas se o ambiente exigir autenticação privada.
- Escolher uma API e um conjunto pequeno de contratos de alto risco para migrar ao staging.
- Revisar as falhas com QA e desenvolvimento e eliminar causas de instabilidade.
- Definir critérios de bloqueio, propriedade e manutenção da suíte.

Resumo em uma frase

Automatizar verificações de API com Python oferece à equipe um caminho leve e reutilizável para obter feedback no ciclo de desenvolvimento; o valor empresarial deve ser confirmado num piloto com serviço, dados e critérios da própria organização.